

```

1  (*
2      CS 51 Lab 19
3      Greenberg-Hastings Cellular Automaton
4      https://en.wikipedia.org/wiki/Greenberg-Hastings_cellular_automaton
5
6  Implementation of the Greenberg-Hastings model described in the
7  article
8
9      Greenberg, J. M., and S. P. Hastings. "Spatial Patterns for
10     Discrete Models of Diffusion in Excitable Media." SIAM Journal on
11     Applied Mathematics 34, no. 3 (1978): 515-23.
12     http://www.jstor.org/stable/2100950.
13 *)
14
15 (*
16     *****
17     WARNING: If you have photosensitive epilepsy, you should avoid
18     viewing the Greenberg-Hastings cellular automaton. It exhibits
19     rapidly flashing visual patterns.
20     *****
21 *)
22
23 module G = Graphics ;;
24
25 (* Automaton parameters *)
26 let cGRID_SIZE = 100 ;;          (* width and height of grid in cells *)
27 let cSPARSITY = 25 ;;            (* inverse of proportion of cells initially live *)
28
29 (* Rendering parameters *)
30 let cCOLOR_LEGEND = G.rgb 173 106 108 ;; (* color for textual legend *)
31 let cSIDE = 8 ;;                 (* width and height of cells in pixels *)
32 let cRENDER_FREQUENCY = 1        (* how frequently grid is rendered (in ticks) *) ;;
33 let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"
34
35 type gh_state =
36   | Resting
37   | Excited
38   | Refractory
39
40 (* offset index off -- Returns the `index` offset by `off` allowing
41    for wraparound. *)
42 let rec offset (index : int) (off : int) : int =
43   if index + off < 0 then
44     cGRID_SIZE - (offset ~-index ~-off)
45   else
46     (index + off) mod cGRID_SIZE ;;
47
48 (* gh_update grid i j -- Returns the updated state for cell at i, j
49    based on the Greenberg-Hastings excitable media rules. *)
50 let gh_update (grid : gh_state array array) (i : int) (j : int) =
51   let neighbors = ref 0 in
52   for i' = ~-1 to ~+1 do

```

```

53   for j' = ~-1 to ~+1 do
54     if not (i' = 0 && j' = 0) then
55       let i_ind = offset i i' in
56       let j_ind = offset j j' in
57       neighbors := !neighbors
58         + match grid.(i_ind).(j_ind) with
59         | Resting -> 0
60         | Refractory -> 0
61         | Excited -> 1
62     done
63   done;
64   match grid.(i).(j) with
65   | Excited -> Refractory
66   | Refractory -> Resting
67   | Resting -> if !neighbors > 0 then Excited else Resting ;;
68
69   (* gh_initialize grid -- Seeds `grid` with a half-line of Excited and
70      Refractory cells at the center column, as per the pattern described
71      in Greenberg & Hastings (1978). This produces the characteristic
72      spiral waves. *)
73   let gh_initialize (grid : gh_state array array) : unit =
74     for i = cGRID_SIZE / 2 to cGRID_SIZE - 1 do
75       grid.(i).(cGRID_SIZE / 2) <- Refractory;
76       grid.(i).(cGRID_SIZE / 2 + 1) <- Excited
77     done
78
79   module GHSpec : (Cellular.AUT_SPEC
80     with type state = gh_state) =
81     struct
82       type state = gh_state
83       let name : string = "Greenberg-Hastings"
84       let initial : state = Resting
85       let grid_size : int = cGRID_SIZE
86       let initialize = gh_initialize
87       let update = gh_update
88       let cell_color (cell : state) : G.color =
89         match cell with
90         | Resting -> G.rgb 245 240 246
91         | Excited -> G.rgb 56 95 113
92         | Refractory -> G.rgb 43 65 98
93       let side_size : int = cSIDE
94       let legend_color : G.color = cCOLOR_LEGEND
95       let font : string option = cFONT
96       let render_frequency : int = cRENDER_FREQUENCY
97     end ;;
98
99   module Aut = Cellular.Automaton (GHSpec) ;;
100
101   (* .....
102      Running the automaton and displaying the results
103      *)
104
105   (* main seed -- Runs the automaton using the provided random `seed`

```

```

106     (for replicability), displaying updates to the graphics window. *)
107 let main seed =
108
109     Random.init seed;      (* initialize randomness *)
110     Aut.graphics_init ();  (* ... and graphics window *)
111
112     Aut.run_grid (Aut.initialized_grid ()) ;;
113
114     (* Run the automaton with user-supplied seed *)
115 let _ =
116     if Array.length Sys.argv <= 1 then
117         Printf.printf "Usage: %s <seed>\n where <seed> is integer seed\n"
118             Sys.argv.(0)
119     else
120         main (int_of_string Sys.argv.(1)) ;;
121

```